Evaluating Subquadratic MST Approximations for Efficient Single-Linkage Clustering

Gavin Gleinig **Brendan Larson**

Abstract

This paper evaluates the application of single-linkage clustering to the Metric Forest Completion (MFC) approximation framework for finding metric Minimum Spanning Trees (MST). We compare MFC-based clusterings against optimal MST, initial k-centering, and other spanning tree variations across both synthetic and real-world datasets. Using Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) metrics, we demonstrate that on synthetic data, the MFC approximation maintains high performance compared to the MST baseline. In real-world evaluations, while single-linkage clustering gave poor performance with the Cooking dataset, the MFC approximation outperformed exact MSTs and k-centering on the GreenGenes dataset. We show that the MFC approximation provides an efficient, subquadratic alternative to exact MSTs for hierarchical clustering.

1 Introduction

Clustering is a fundamental machine learning task that involves partitioning objects into groups, or clusters, so that members of the same set are more closely related to one another than to those in any other group. Clustering has many applications, including anomaly detection, image segmentation, and genomics [4]. One simple approach is single-linkage clustering, which organizes objects into a hierarchical classification by repeatedly combining clusters that contain the closest pair of elements not yet belonging to the same cluster. Once a hierarchy is found, it can then be "cut" at different levels to produce a specific number of clusters.

It is well established that single link clustering is functionally equivalent to finding a Minimum Spanning Tree (MST) on a complete graph [3]. In a metric space, data can be represented as a complete graph of n points where edge weights represent pairwise distances. However, finding an exact Metric MST requires computing every pairwise distance and running a greedy algorithm such as Kruskals, which takes O($n^2 \log n$) time. Because calculating complex distance metrics can be computationally expensive, finding an exact MST to perform exact single-linkage clustering is challenging for massive datasets.

This challenge motivated the Metric Forest Completion (MFC) framework by Veldt et al [6]. The MFC framework is an approximation algorithm for the Metric MST problem that first finds a forest of disconnected components using practical heuristics, and then finds a small-weight set of edges to connect disjoint components of the forest into a spanning tree. Veldt et al. show that the MFC framework achieves good spanning tree approximations on synthetic and real datasets. This approximation holds a theoretical guarantee of being a 2γ approximation of the optimal MST, where γ represents how close the initial forest is to the optimal MST [5].

While the MFC framework has been experimentally shown to produce great spanning tree approximations, the viability of these spanning trees for downstream clustering tasks requires more investigation. In this project, we evaluate the MFC framework's clustering performance against exact MST and k-centering baselines.

Testing across both synthetic Gaussian data and real-world datasets (Cooking and GreenGenes), we use ARI and NMI metrics to quantify cluster quality. We found that the MFC approximation handles high cluster counts well on synthetic data in many dimensions. Additionally, our real-world evaluations indicate that, for tasks where single-linkage clustering is appropriate, the MFC-Approx algorithm can outperform exact MSTs with a fraction of the computational time.

2 Preliminaries

In this section we provide a technical background and establish the terminology and notation used throughout this project.

We notate a finite metric space as $(\mathcal{X}, d)$ where $\mathcal{X} = \{x_1, x_2, ..., x_n\}$ is a set of data points and the distance between the ith and jth points is defined by $d(x_i, x_j)$. This space is associated with an implicit complete graph $G_{\mathcal{X}} = (\mathcal{X}, E_{\mathcal{X}})$ where $\mathcal{X}$ is the node set and the edge set $E_{\mathcal{X}} = \binom{\mathcal{X}}{2}$ includes all pairs of nodes.

An initial clustering step produces a partitioning $\mathcal{P} = \{P_1, P_2, ..., P_t\}$ such that the entire dataset is covered ($\mathcal{X} = \bigcup_{i=1}^{t} P_i$) and all components are disjoint ($P_i \cap P_j = \emptyset$ for $i \neq j$). Each component P_i has an internal partition spanning tree noted as $T_i = (P_i, E_{T_i})$, and the union of these trees forms the initial forest. This initial forest is represented as the graph $G_t = (\mathcal{X}, E_t)$ where the full edge set is $E_t = \bigcup_{i=1}^{t} E_{T_i}$.

The Metric Forest Completion problem seeks to connect the disjoint components of the initial forest into a single spanning tree. We let $\mathcal{I} = \{(x, y) \in \mathcal{X} \times \mathcal{X} : P(x) \neq P(y)\}$ represent the set of all possible inter-component edges. The MFC objective is to find a valid completion set of edges $M \subseteq \mathcal{I}$ that minimizes the total weight of the final spanning tree. The objective function is:

$$\begin{aligned} \text{minimize} \quad & w_{\mathcal{X}}(M) + w_{\mathcal{X}}(E_t) \\ \text{subject to} \quad & M \text{ completes } \mathcal{P}. \end{aligned}$$

To convert the resulting spanning tree $T = (\mathcal{X}, E_{\mathcal{X}})$ into clusters, we use a standard single-linkage approach. By removing all edges with a weight larger than a chosen distance threshold τ, the remaining connected components form our resulting single-linkage clusters.

3 Background

In this section, we cover the computational bottlenecks of finding an exact MFC solution and state the formal theoretical guarantees provided by the MFC-Approx algorithm introduced by Veldt et al.

Solving the MFC objective optimally requires finding the minimum distance between every pair of disjoint components to form an exact coarsened graph. Veldt et al. prove that calculating these exact pairwise distances has a lower bound complexity of $\Omega(n^2)$.

To bypass the $\Omega(n^2)$ bottleneck, the MFC framework uses a subquadratic approximation algorithm (MFC-Approx). Rather than evaluating all points across partitions, the algorithm selects an arbitrary representative point $s_i \in P_i$ for each component. It then connects partitions by computing the minimum distance between each partition's points and the representative node of the opposing partition. It will connect these found minimum nodes as the edges for the coarsened graph.

Veldt et al showed that this approximation has several theoretical guarantees. First, they prove that MFC-Approx has a subquadratic runtime, running in $\tilde{O}(ntQ_x + t^2)$ time, where $\tilde{O}$ hides a logarithmic term and Q_X is the complexity for one distance query. Assuming $Q_X = \tilde{O}(1)$, then this algorithm is subquadratic in memory, query, and runtime complexity provided $t = o(n)$. Second, Veldt et al. proves that constructing the final spanning tree from the graph given by MFC-Approx is a 2-approximation for the optimal MFC objective.. These proofs are given in [5].

The quality of the final metric spanning tree depends on the structuring quality of the initial clustering forest. We use a γ-overlap metric to measure the quality of the initial forest. Let $T_{\mathcal{X}}$ represent the set of minimum spanning trees of $G_{\mathcal{X}}$, and ($T \in \mathcal{T}_{\mathcal{X}}$) denote an arbitrary MST. Also let $T(P_i)$ denote the induced subgraph of the optimal tree T strictly on the points inside component P_i, and

$T(\mathcal{P}) = \bigcup_{i=1}^{t} T(P_i)$ denote the set of all edges from the optimal MST T that are contained entirely inside the components of $\mathcal{P}$. We define γ-overlap as

$$\gamma(\mathcal{P}) = \frac{w_{\mathcal{X}}(E_t)}{\max_{T \in \mathcal{T}_{\mathcal{X}}} w_{\mathcal{X}}(T(\mathcal{P}))}$$

This ratio compares the weight of the edges that the initial forest places inside the components ($\mathcal{P}$) against the maximum weight of edges any true optimal spanning tree would place inside those same components. By definition, γ is always ≥ 1, and lower values indicate a higher quality initial clustering. This γ overlap is used to dictate the approximation guarantees of the MFC-approx when approximating Metric MST, with MFC-approx being a 2γ approximation of Metric MST.

4 Experimental Results

In this section, we evaluate the clustering ability and computational efficiency of spanning trees produced by the MFC framework. We compare the MFC framework against exact and naive baselines across both synthetic and real-world datasets.

4.1 Experimental Setup

All dataset generation, parsing, algorithms, and experiments were written in the Julia programming language. Synthetic data experiments were run on 32 GB of RAM and an AMD RYZEN 7000 series processor. Real-world evaluations were run on a system with 16GB of RAM and an 11th Gen Intel Core i7 processor. Runtime results and commentary are given in the Appendix.

Given a set of data points, our pipeline begins with an initial rough clustering using a 2-approximate k-centering algorithm. Initial forests are then found on each partition using Kruskal's algorithm. To connect these disjoint forests into a single spanning tree, we evaluate and compare the following completion algorithms:

MFC-Optimal: This variant tests the optimal solution to the MFC objective. It iterates through every node in every partition to find the absolute minimum distance between all pairs of disjoint components.

MFC-Approx: This is the subquadratic approximation algorithm outlined in Section 3. It selects an arbitrary representative node for each component and only calculates distances between that representative and the points in other partitions.

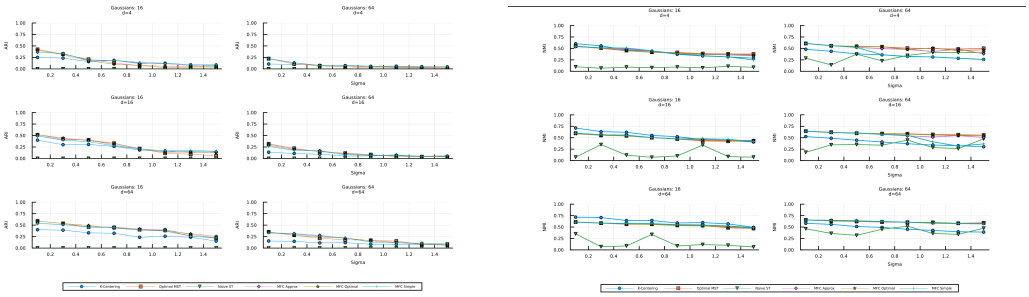
MFC-Simple: This completion algorithm connects an arbitrary representative node from each component to each other component.

We benchmark the trees derived from the MFC framework against a minimum spanning tree and a naive spanning tree baseline. The exact MST is found on the complete graph using Kruskal's algorithm. The naive spanning tree is created by randomly selecting edges from the complete graph, adding them to the spanning tree only if they do not form a cycle.

To convert these resulting spanning trees into clusterings, we sweep through a range of distance thresholds (τ), deleting all edges larger than the threshold. We report the best-case performance for each algorithm by selecting the threshold that produces the highest Adjusted Rand Index (ARI) score and Normalized Mutual Information (NMI) score. The Adjusted Rand Index measures the similarity between our predicted clusters and the ground-truth labels by evaluating all pairwise agreements, correcting for groupings that occur by random chance. Normalized Mutual Information quantifies the amount of shared information between the true and predicted clusterings, scoring from 0 to 1, where higher values indicate stronger structural alignment.

4.2 Datasets

We evaluate our methods on datasets with known ground-truth labels. For synthetic data, datasets are made up of $N = 20,000$ points sampled from Gaussian distributions. We vary dimensionality $d \in \{4, 16, 32, 128\}$, variance $\sigma \in \{.1, .3, ..., 1.5\}$, and mean (from -5 to 5). Ground truth labels were assigned based on the source Gaussian of each generated point.



(a) ARI vs Sigma

(b) NMI vs Sigma

Figure 1: Cluster Metrics on Synthetic Gaussians with Varying Variance

For real-world datasets, we ran evaluations on the Cooking [1] and GreenGenes [2] datasets. In the Cooking dataset, objects represent sets of recipe ingredients, with 20 ground truth cuisine categories. Sets are compared with a Jaccard distance metric. GreenGenes is a chimera-checked 16S megagenomic sequence dataset. We evaluate the GreenGeenes-Aligned variation, consisting of strings of length 7682, using a Hamming distance to compare sequences.

4.3 Synthetic Data Evaluation

Our primary objective with our synthetic dataset was to explore how the MFC framework performed on different levels of spatial complexity and cluster counts. We evaluated the algorithms on generated datasets consisting of different numbers of Gaussian distributions $g = \{16, 32, 64, 128, 256\}$, varying dimensionality, at different variance levels. We then measured clustering accuracy using ARI and NMI metrics. An insightful selection of these results $g = \{16, 64\}$ and $d = \{4, 16, 64\}$ are produced in Figure 1. For the full results of these test refer to the Appendix with ARI in 4 and NMI in 3.

Overall, we found that dimensionality had little impact on clustering accuracy across all tested methods once it passed a certain threshold. Specifically, clustering NMI scores plateaued at d = 16; increasing dimensions from 16 to 64 made only minute impact on the scores of any algorithm. Furthermore, as the number of dimensions increased, all single-linkage clustering approaches (MST and MFC variants) performed similarly.

Across all tests, clustering accuracy measured by ARI was significantly lower than when measured by NMI. Despite the overall lower scores, single-linkage clustering generally outperformed k-centering across all tested situations. Specifically, as the number of underlying Gaussians increased (e.g., at g = 64 and 256), ARI scores universally dropped, often falling below 0.3. This low score is likely due to the spatial overlap of points from different distributions, which mixes cluster boundaries. Even so, all MFC variants performed better than k-centering in these dense environments, with scores only slightly below the exact MST baseline. In contrast, when the number of clusters was decreased to $g = 16$, all clustering algorithms performed much better on ARI metrics. For example, at a dimensionality of 64, the exact MST scored 0.5890, and MFC-Approx scored a very close 0.5397. This confirmed our theory that this spatial overlap was the primary cause of low ARI, which is supported by Appendix data showing that ARI improves as the mean distance between cluster centers increases.

When evaluating clustering algorithms with NMI, it was found that as the number of ground truth Gaussians increases, the MFC variant algorithms and MST algorithms perform better than simple k-centering. At a low number of ground truth clusters (g = 16), k-centering actually outperformed all single linkage clustering approaches across all dimensions. But, at g = 64, we observed a distinct tipping point where single-linkage clustering begins to consistently outperform k-centering. For example, at greater numbers of ground truth Gaussians, the MFC and MST algorithms outperformed k-centering on average by 0.2945 (MST) and 0.2297 (MFC-Approx). We also observed that as the number of Gaussians increases, the performance separation between the different MFC variations becomes more pronounced, though they eventually converge at higher dimensions.

This k-centering advantage at low cluster counts holds true until a sigma of 1.0, when k-centering begins to perform worse than the single linkage variants. However, we did note a specific edge case: at low dimensions (d = 4) and high variance (sigma of 1.1+), MFC-Approx begins to lose to both

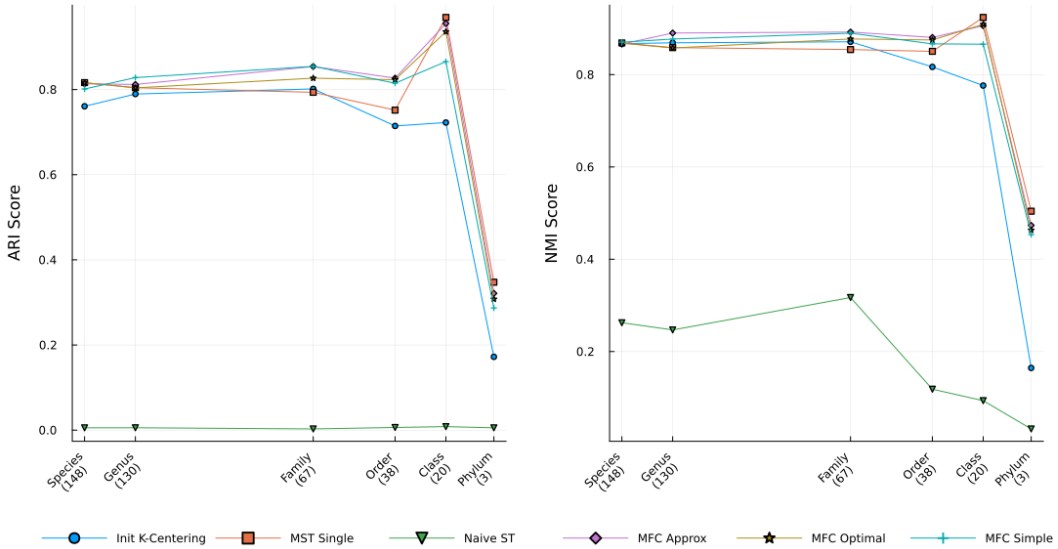


Figure 2: Mean Cluster Metrics for Each Taxonomic Level

exact MST and MFC-Optimal. Otherwise, MFC variants and MST are much more robust and do not appear heavily affected by varying sigma, even up to a sigma of 1.5, whereas k-centering is susceptible to bad starting points when running.

Ultimately, we expect the MFC framework approximations to fall somewhere between the initial k-centering and the exact MST clustering approaches. However, in almost all cases tested, the MFC algorithms heavily favored the exact MST approaches, performing nearly as well as the optimal MST. These results further provide support for the high approximation quality and runtime improvements provided by MFC Approx and even MFC Simple over an exact MST.

Regarding computational efficiency, the runtime increased as the dimensionality grew, as expected. See Table 3 in the Appendix for details. Overall, the runtime of these algorithms increases with the dimensionality of the data set; however, for the MFC set of problems, echoing Veldt et al.'s results, as the amount of initial Gaussians increases, runtime decreases. This result is due to running each partition MST on less nodes. K-centering remained quick to compute even as the dimensionality increased. For the runtime of the enitre pipeline, the runtime of the threshold sweep was excluded. This is due to the threshold sweep having negligible effects on the runtime, and if included, would only increase the runtime of MST and MFC by a small factor. Further, this threshold sweep was done by evaluating each threshold against the greatest found ARI, which requires already knowing the clusters, which would likely not be done in practice.

4.4 Real-World Data Evaluation

We evaluated the ability of MFC to effectively cluster two real-world datasets. For both datasets, we reproduce the runtime and overlap bound results from previous works (detailed in the Appendix).

For the Cooking dataset, we found that Jaccard distance and single-linkage-based clustering will not meaningfully cluster recipes into ground truth cuisines. All methods, including the MST baseline, produced an ARI of 0.0 and near-zero NMI scores. Upon further investigation, we found that, out of 39,773 total edges in the minimum spanning tree, 15,117 edges connected recipes from different ground truth cuisines. We suspect this clustering failure is due to the presence of universal ingredients such as water, salt, and eggs in small recipes bringing cuisines close together under the Jaccard distance. As single linkage clustering combines two clusters based on a single closest pair of points between them, these small recipes and common ingredients cause this clustering method to fail.

With the GreenGenes dataset, we also test how the different spanning trees produced by the MFC framework fare. For our primary evaluation, we sampled $N = 3,000$ strings. Because the dataset features hierarchical taxonomic labels, this sample provides ground-truth cluster counts ranging from 3 (Phylum level) to 148 (Species level). For each taxonomic level, we run five independent tests

across all clustering variations, reporting the mean ARI and NMI scores. MFC variants were created with a number of initial components equal to the number of ground truth clusters. These results are presented in Figure 2. We notice that, across all taxonomic levels, single-linkage-based approaches outperform the initial 2-approximate k-centering clustering. This performance difference is smaller at the Species level with more ground truth clusters and larger at the Class level.

When comparing the MFC framework to an exact MST, we notice a difference in performance based on the granularity of ground truth. At the highest taxonomic levels with only a few ground truth clusters (Phylum, Class), an exact MST outperforms the MFC variants. However, at more mid-level taxonomies (Order, Family, Genus), the MFC framework approaches overtake exact MST in both ARI and NMI. We suspect this may be due to robustness in the MFC framework to noise in the data. Single linkage clustering is known to struggle with noise points, causing bridges between clusters. Perhaps by coarsening the graph and limiting sub-MSTs, the MFC framework ensures these noise points don't have an effect until later.

Finally, when evaluating variations of the MFC framework, we note that MFC-Approx consistently gives slightly better cluster metrics than MFC-Optimal. Given the computational speedup of using the approximate completion step over optimally solving the MFC objective, this further motivates the use of MFC-Approx beyond the proven theoretical bounds.

5 Conclusion and Discussion

In this work, we evaluated the Metric Forest Completion (MFC) framework as a subquadratic alternative to exact Metric Minimum Spanning Trees (MST) for hierarchical clustering. By refining initial k-centering forests through the MFC framework and applying single-linkage clustering, we demonstrated that MFC effectively bypasses the $\Omega(n^2)$ computational bottleneck of exact MSTs.

Our evaluations on synthetic data gave us insights into the behavior of the MFC framework across varying spatial complexities. Most notably, we found that MFC-Approx consistently achieved near-parity with the exact MST across almost all test cases, proving it as an efficient alternative to MST clustering that bypasses the quadratic distance calculations.

We also evaluated these frameworks on real-world data, including a cooking dataset and gene sequences. On the cooking dataset, we observed that Jaccard distance does not provide meaningful cluster boundaries for single-linkage methods. However, on the GreenGenes dataset, MFC single-linkage clustering consistently outperformed the initial k-centering forests. This suggests that practitioners should utilize the MFC framework to achieve superior structural approximations when a slight increase in computation time is acceptable. For all tests, we have reproduced runtime results as observed in previous works.

The limitations of our work are that we only used one type of MST-based clustering, arguably the simplest one. Further, our work only considers graphs up to 30,000 points in the GreenGenes graph; results may vary when running these algorithms on datasets with significantly more edges.

Continuation of this research could be to evaluate other MST-based clustering methods, such as Genie Clustering [4]. This clustering pipeline could further be optimized through the use of parallelism, especially in finding the minimum spanning trees of each sub-cluster. Overall, we contribute practical evaluations proving the effectiveness of the MFC approximation algorithm for metric MST clustering, alongside a native Julia implementation of the MFC framework.

References

[1] Ilya Amburg, Nate Veldt, and Austin Benson. Clustering in graphs and hypergraphs with categorical edge labels. In *Proceedings of The Web Conference 2020*, WWW '20, page 706–717, New York, NY, USA, 2020. Association for Computing Machinery.

[2] Todd Z DeSantis, Philip Hugenholtz, Neils Larsen, Mark Rojas, Eoin L Brodie, Keith Keller, Thomas Huber, Daniel Dalevi, Ping Hu, and Gary L Andersen. Greengenes, a chimera-checked 16s rrna gene database and workbench compatible with arb. *Applied and environmental microbiology*, 72(7):5069–5072, 2006.

[3] J. C. Gower and G. J. S. Ross. Minimum spanning trees and single linkage cluster analysis. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 18(1):54–64, 1969.

[4] A. K. Jain, M. N. Murty, and P. J. Flynn. Data clustering: a review. *ACM Comput. Surv.*, 31(3):264–323, September 1999.

[5] Nate Veldt, Thomas Stanley, Benjamin W. Priest, Trevor Steil, Keita Iwabuchi, T. S. Jayram, Grace J. Li, and Geoffrey Sanders. Better learning-augmented spanning tree algorithms via metric forest completion, 2026.

[6] Nate Veldt, Thomas Stanley, Benjamin W. Priest, Trevor Steil, Keita Iwabuchi, T.S. Jayram, and Geoffrey Sanders. Approximate forest completion and learning-augmented algorithms for metric minimum spanning trees. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML'25. JMLR.org, 2025.

Table 1: Evaluation Results on Cooking dataset ($N = 39,774$; γ-overlap Bound = 1.994).

Algorithm	Runtime (s)	ARI	NMI	Cost Ratio
Initial k-Centering	2.75	0.0199	0.0871	–
Optimal MST	1861.19	0.0000	0.3956	1.0000
MFC Approximation	203.01	0.0000	0.3956	1.0475
MFC Optimal	1704.76	0.0000	0.3956	1.0471
MFC Simple	214.10	0.0000	0.3956	–
Naive Spanning Tree	–	0.0000	0.3951	–

A Appendix

A.1 Cooking Dataset Reproduction

While we found in the main report that a Jaccard distance was not able to successfully cluster ground truth cuisines, we provide our full results, including runtime results, in Figure 1.

A.2 Naively Creating Clusters from Spanning Trees with GreenGenes

In the main results, we threshold each spanning tree to find the best possible ARI and NMI scores. However, deleting the k longest edges also gives k+1 ground truth clusters. To evaluate the effectiveness of this simple approach on real-world datasets, we took ground truth taxonomies at each level, and attempted to create a number of clusters equaling the ground truth number of clusters at each level. Our results are presented in Figure 2.

This test resulted in worse cluster quality metrics than the k-centering algorithm for most levels. With this data, it appears that k distinct clusters are *not* separated by the k-1 absolute longest edges. Noise points and local density likely constrain this naive single link clustering.

By not checking every possible threshold against a ground truth and taking the best possible clustering, the issues behind basic single link clustering become clear.

A.3 Minimum Spanning Trees for Isolated Clustering

We also explored single link clustering on isolated clustering by generating gausians with mean (-200, 200) on n=20,000 and d=256, g=128. we found that single link does not do better than k-centering when dealing with isolated clustering. This is likely due to k-centering innate advantages inside this scenario. Further single link with best threshold may be unable to separate the correct threshold because it may separate clusters that formed near to each other.

A.4 Full Sweep Synthetic results

In the main results we mention testing gaussians ranging form $g = \{16, 32, 64, 128, 256\}$ over dimensions of $d = \{4, 16, 64, 128\}$, however we only show the most impactful $g = \{16, 64\}$ and $d = \{4, 16, 64\}$. Here in the Appendix, we demonstrate the full range of data for NMI in 3 and the full range for ARI in 4. The results show that the trend of MFC variants performing better at identifying the structure begins at $g = 32$ for NMI and continues to degrade for ARI as gaussian increase.

A.5 Average Synthetic runtime

In this section, we found that sigma did not greatly affect runtime, so the variations in sigma for runtime were averaged together in 3. This table shows that as the amount of gaussian increases, the runtime of the MFC approximations improves; however, as dimensionality increases, all runtimes increase. k-centering further appears to be relatively fast overall for Gaussian and dimensions.

Hyperparams for below table: - Dataset size = 30k objects - 20 ground truth clusters used for MFC Approx - k-centering is equal to k for each level - cut k-1 links to get k clusters

Table 2: Naive Single-Linkage Clustering Results ($N = 30,000$)

Level (True Clusters)	Algorithm	ARI	NMI
Phylum ($k = 52$)	Optimal MST	0.4199	0.5564
	MFC Approximation	0.4265	0.5729
	Initial k-Centering	0.4117	0.5597
Class ($k = 153$)	Optimal MST	0.2586	0.5484
	MFC Approximation	0.2782	0.5579
	Initial k-Centering	0.7384	0.7939
Order ($k = 291$)	Optimal MST	0.2660	0.5922
	MFC Approximation	0.3077	0.6353
	Initial k-Centering	0.7016	0.8183
Family ($k = 500$)	Optimal MST	0.5925	0.7862
	MFC Approximation	0.6375	0.7917
	Initial k-Centering	0.5797	0.8313
Genus ($k = 906$)	Optimal MST	0.6568	0.8102
	MFC Approximation	0.6974	0.8130
	Initial k-Centering	0.7778	0.8613

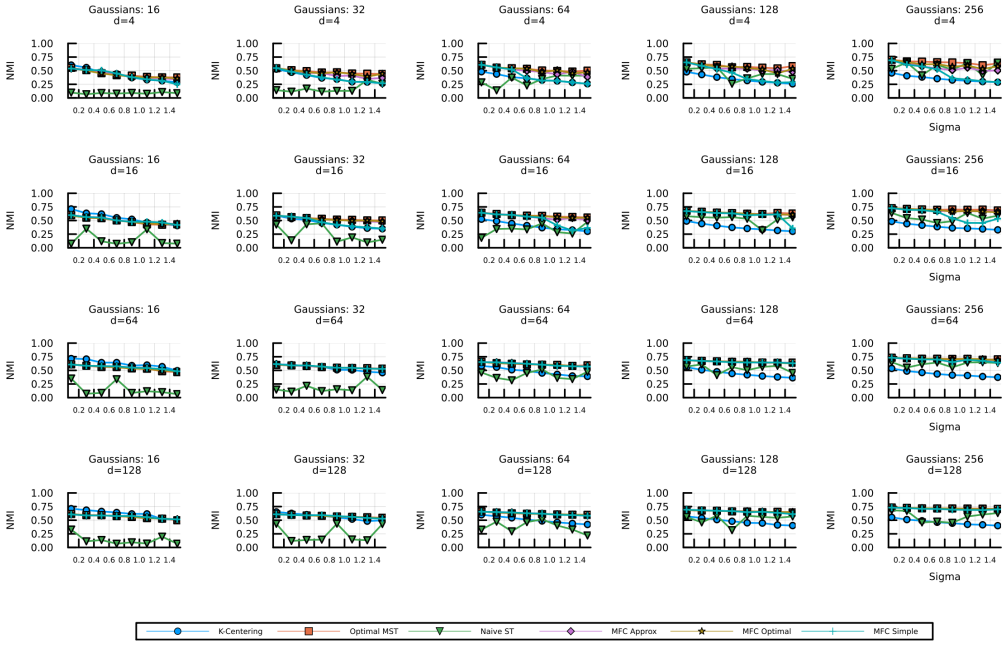


Figure 3: Full sweep of clustering NMI

Table 3: Average runtime (s) across dimensions and Gaussian counts n= 20,000

Synthetic data at $d = 4$ Runtime (s)					
Algorithm	g=16	g=32	g=64	g=128	g=256
K-Means	0.108305	0.170226	0.358709	0.722386	0.798351
K-Centering	0.00479016	0.00908196	0.0186155	0.0383299	0.0743805
Optimal MST	18.0093	17.0229	19.0539	20.7654	16.4493
MFC Approximation	4.23537	2.33012	1.91532	1.21963	0.55219
MFC Optimal	5.13144	3.4828	3.34323	2.83446	1.81107
MFC Simple	4.03079	2.42291	1.94108	1.0957	0.445474
Naive Spanning Tree	0.00328555	0.00273597	0.00360704	0.00483617	0.00277489
Synthetic data at $d = 16$ Runtime (s)					
Algorithm	g=16	g=32	g=64	g=128	g=256
K-Means	0.15064	0.230648	0.410681	0.622767	0.928364
K-Centering	0.00952554	0.0183342	0.0368887	0.0719186	0.147331
Optimal MST	17.9089	17.9333	17.6163	17.7157	17.7714
MFC Approximation	3.79767	2.18003	1.62915	1.25516	0.735585
MFC Optimal	5.99186	4.62655	4.2774	4.04338	3.50229
MFC Simple	3.80326	2.16351	1.60287	1.2205	0.626842
Naive Spanning Tree	0.00376019	0.00403799	0.00355045	0.00359993	0.00356155
Synthetic data at $d = 64$ Runtime (s)					
Algorithm	g=16	g=32	g=64	g=128	g=256
K-Means	0.145453	0.191953	0.2999	0.507181	0.87472
K-Centering	0.0296561	0.0566331	0.114104	0.223161	0.446147
Optimal MST	23.786	23.7467	23.3689	22.919	22.8927
MFC Approximation	4.89265	3.59962	2.2732	1.9729	1.46046
MFC Optimal	12.1592	11.4321	10.6478	10.3383	10.1169
MFC Simple	4.981	3.64008	2.22964	1.90691	1.20493
Naive Spanning Tree	0.0049028	0.00476591	0.0046767	0.00464264	0.0044869
Synthetic data at $d = 128$ Runtime (s)					
Algorithm	g=16	g=32	g=64	g=128	g=256
K-Means	0.109418	0.224384	0.455735	0.712585	0.829152
K-Centering	0.0645835	0.13885	0.241543	0.484963	0.991533
Optimal MST	33.2052	32.8471	33.0391	33.1525	33.5479
MFC Approximation	6.67293	4.82138	3.59927	2.99692	2.88692
MFC Optimal	21.2357	20.5341	20.2193	19.929	20.2722
MFC Simple	6.65866	4.789	3.51439	2.74085	2.26454
Naive Spanning Tree	0.00675932	0.00671693	0.00675219	0.00673322	0.007208

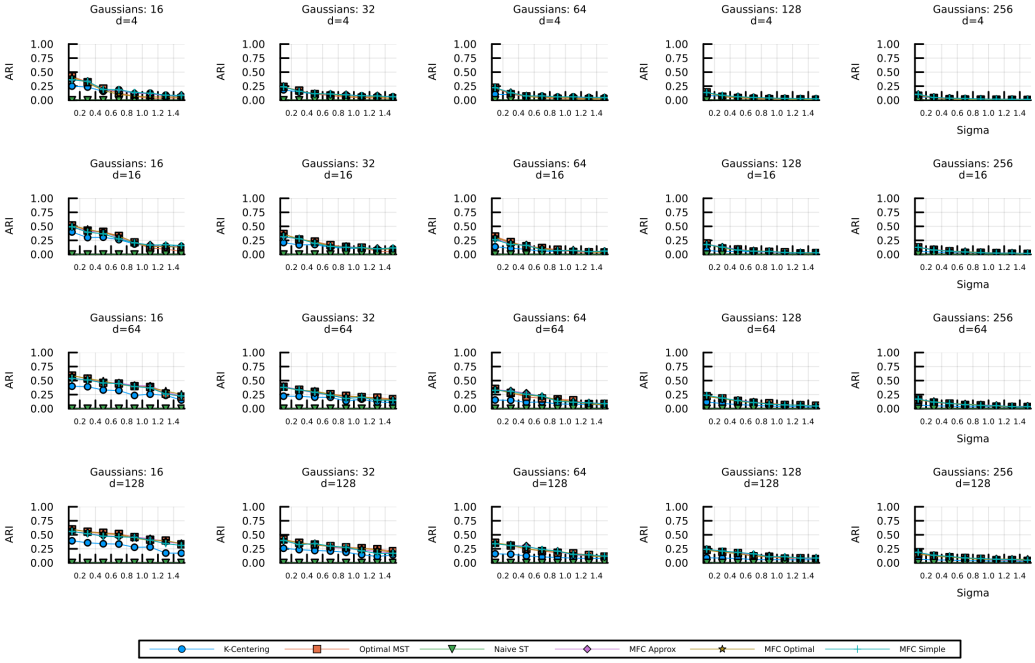


Figure 4: Full sweep of clustering ARI